

Snapshot Read - Engine Read Path and Transaction API

In this chapter, you will:

- Finish the read path based on previous chapter to support snapshot read.
- Implement the transaction API to support snapshot read.
- Implement the engine recovery process to correctly recover the commit timestamp.

At the end of the day, your engine will be able to give the user a consistent view of the storage key space.

During the refactor, you might need to change the signature of some functions from `&self` to `self: &Arc<Self>` as necessary.

To run test cases,

```
cargo x copy-test --week 3 --day 3
cargo x scheck
```

Note: You will also need to pass test cases for 2.5 and 2.6 after finishing this chapter.

Task 1: LSM Iterator with Read Timestamp

The goal of this chapter is to have something like:

```
let snapshot1 = engine.new_txn();
// write something to the engine
let snapshot2 = engine.new_txn();
// write something to the engine
snapshot1.get(/* ... */); // we can retrieve a consistent snapshot of a
previous state of the engine
```

To achieve this, we can record the read timestamp (which is the latest committed timestamp) when creating the transaction. When we do a read operation over the transaction, we will only read all versions of the keys below or equal to the read timestamp.

In this task, you will need to modify:

```
src/lsm_iterator.rs
```

To do this, you will need to record a read timestamp in `LsmIterator`.

```
impl LsmIterator {
    pub(crate) fn new(
        iter: LsmIteratorInner,
        end_bound: Bound<Bytes>,
        read_ts: u64,
    ) -> Result<Self> {
        // ...
    }
}
```

And you will need to change your LSM iterator `next` logic to find the correct key.

Task 2: Multi-Version Scan and Get

In this task, you will need to modify:

```
src/mvcc.rs
src/mvcc/txn.rs
src/lsm_storage.rs
```

Now that we have `read_ts` in the LSM iterator, we can implement `scan` and `get` on the transaction structure, so that we can read data at a given point in the storage engine.

We recommend you to create helper functions like `scan_with_ts(/* original parameters */, read_ts: u64)` and `get_with_ts` if necessary in your `LsmStorageInner` structure. The original `get/scan` on the storage engine should be implemented as creating a transaction (snapshot) and do a `get/scan` over that transaction. The call path would be like:

```
LsmStorageInner::scan -> new_txn and Transaction::scan ->
LsmStorageInner::scan_with_ts
```

To create a transaction in `LsmStorageInner::scan`, we will need to provide a `Arc<LsmStorageInner>` to the transaction constructor. Therefore, we can change the signature of `scan` to take `self: &Arc<Self>` instead of simply `&self`, so that we can create a transaction with `let txn = self.mvcc().new_txn(self.clone(), /* ... */)`.

You will also need to change your `scan` function to return a `TxnIterator`. We must ensure the snapshot is live when the user iterates the engine, and therefore, `TxnIterator` stores the snapshot object. Inside `TxnIterator`, we can store a `FusedIterator<LsmIterator>` for now. We will change it to something else later when we implement OCC.

You do not need to implement `Transaction::put/delete` for now, and all modifications will still go through the engine.

Task 3: Store Largest Timestamp in SST

In this task, you will need to modify:

```
src/table.rs  
src/table/builder.rs
```

In your SST encoding, you should store the largest timestamp after the block metadata, and recover it when loading the SST. This would help the system decide the latest commit timestamp when recovering the system.

Task 4: Recover Commit Timestamp

Now that we have largest timestamp information in the SSTs and timestamp information in the WAL, we can obtain the largest timestamp committed before the engine starts, and use that timestamp as the latest committed timestamp when creating the `mvcc` object.

If WAL is not enabled, you can simply compute the latest committed timestamp by finding the largest timestamp among SSTs. If WAL is enabled, you should further iterate all recovered memtables and find the largest timestamp.

In this task, you will need to modify:

```
src/lsm_storage.rs
```

We do not have test cases for this section. You should pass all persistence tests from previous chapters (including 2.5 and 2.6) after finishing this section.

Test Your Understanding

- So far, we have assumed that our SST files use a monotonically increasing id as the file name. Is it okay to use `<level>_<begin_key>_<end_key>_<max_ts>.sst` as the SST file name? What might be the potential problems with that?
- Consider an alternative implementation of transaction/snapshot. In our implementation, we have `read_ts` in our iterators and transaction context, so that the user can always access a consistent view of one version of the database based on the timestamp. Is it viable to store the current LSM state directly in the transaction context in order to gain a consistent snapshot? (i.e., all SST ids, their level information, and all memtables + ts) What are the pros/cons with that? What if the engine does not have memtables? What if the engine is running on a distributed storage system like S3 object store?

- Consider that you are implementing a backup utility of the MVCC Mini-LSM engine. Is it enough to simply copy all SST files out without backing up the LSM state? Why or why not?

We do not provide reference answers to the questions, and feel free to discuss about them in the Discord community.

Your feedback is greatly appreciated. Welcome to join our [Discord Community](#).

Found an issue? Create an issue / pull request on github.com/skyzh/mini-lsm.

mini-lsm-book © 2022-2025 by Alex Chi Z is licensed under CC BY-NC-SA 4.0.